CLAUDE CODE ULTIMATE GUIDE

Leading a Team in the Age of AI

Metrics, adoption, and governance for engineering managers

Florian Bruniaux

October 2026

v1.4.0

Guide v3.44.1

Table of contents

1 The measurement problem 5
1.1 What Anthropic data confirms 5
2 The DORA foundation 6
2.1 Deployment frequency 6
2.2 Change lead time 6
2.3 Change failure rate 6
2.4 Failed deployment recovery time 7
2.5 Deployment rework rate 7
2.6 From four keys to five metrics 7
3 Beyond DORA: The SPACE framework 9
3.1 Satisfaction and Well-being 9
3.2 Performance 9
3.3 Activity 9
3.4 Communication and collaboration 10
3.5 Efficiency and flow 10
3.6 The velocity trap 10
4 AI-specific metrics 11
4.1 % AI-assisted code 11
4.2 CFR: AI code vs. manual code 11
4.3 Review time: AI PRs vs. manual PRs 11
4.4 Developer code understanding 12
4.5 PR comprehension time 12
5 Agentic metrics: What DORA doesn’t measure 13
5.1 Why standard DORA is insufficient for agent workflows 13
5.2 Group 1: RCT-Verifiable metrics 13
5.3 Group 2: Agentic pipeline metrics 14
5.4 Test a harness before standardizing it 15
5.5 Group 3: Agent governance metrics 15
5.6 The heavy-user review time contradiction 16
5.7 The Uplevel Copilot study 16
5.8 pass^k for non-deterministic tests 16
6 Product metrics 18
6.1 Time-to-value 18
6.2 Feature adoption rate 18
6.3 Bug escape rate 18
6.4 Feature CSAT 19
7 By team size 20
7.1 5-person team 20
7.2 25-person team 20
8 Vanity metrics to drop 22
9 The 4-question test 23

9.1 Admit work at verification capacity . . . 23
10 Tooling . . . 24
10.1 Broader delivery intelligence platforms . . . 25
10.2 AI-Generated board narratives . . . 26
11 Probabilistic delivery forecasting . . . 27
11.1 Why a point estimate misleads . . . 27
11.2 Tools . . . 27
11.3 What this does not fix . . . 27
12 Implementation roadmap . . . 29
12.1 Phase 1: Instrument DORA . . . 29
12.2 Phase 2: Baseline and targets . . . 29
12.3 Phase 3: Product and AI metrics . . . 29
13 Reporting delivery capacity to a skeptical board . . . 30
13.1 The diagnostic: Doubt is a trust problem, not a data problem . . . 30
13.2 Bring scenarios, not a velocity chart . . . 30
13.3 Cap strategic objectives, don’t roadmap every feature . . . 31
13.4 Separate committed scope from best-effort scope, and track the hit rate . . . 31
13.5 Align stakeholders before the room, not in it . . . 31
13.6 Where forecasting and AI narratives fit, and where they don’t . . . 32
14 Summary: The complete stack . . . 33
14.1 PR audit trail . . . 33
15 See also . . . 34
16 The series . . . 35

📌 **Who is this whitepaper for?**

Primary audience: Engineering Managers, Tech Leads, CTOs

Prerequisites: Claude Code in active use by the team

Reading time: 25 min

Key takeaways:

- The 5 current DORA metrics recalibrated for an AI context
- The 5 AI-specific signals that standard frameworks don't cover, plus the agentic metrics DORA misses entirely
- The minimal stack by team size: 5 tools for 5 people, not 20
- The 6 vanity metrics to drop starting tomorrow

In 5 lines: Activity counts can rise without improving accepted delivery, maintainability or learning. This whitepaper combines DORA, SPACE and agent-specific measures. Establish a local baseline, distinguish observations from causal explanations, and include human review, rework and cost per accepted task.

1 The measurement problem

AI adoption can change how work is produced and reviewed. Compare similar tasks and services before attributing a change in throughput to the assistant. More commits or generated lines can coexist with slower accepted delivery.

Activity measures can change quickly; quality and maintainability often require longer observation. Track accepted outcomes and human effort rather than assuming generated volume proves improvement.

This guide gives engineering managers, tech leads, and CTOs a practical metrics stack: the DORA foundation recalibrated for an AI context, human factors through SPACE, and the AI-specific signals that standard frameworks don't cover.

1.1 What Anthropic data confirms

Anthropic reported **+67% PRs merged per engineer per day** in January 2026 and 70 to 90% AI-assisted shipped code in an earlier internal study. These figures describe Anthropic's own environment. They are useful context, not an independent benchmark or a target for another team.

Compare one service against its own pre-adoption baseline. If coding time falls but change lead time does not, the constraint is likely in review, validation, or deployment rather than code generation.

2 The DORA foundation

DORA (DevOps Research and Assessment) measures the health of a software delivery system, not individual contributors. The current DORA guide, updated January 5, 2026, defines five metrics grouped into throughput and instability. Apply them to one application or service at a time.

2.1 Deployment frequency

What it measures: How often you deploy to production.

How to measure: Count production deployments per day, week, or month. Most CI/CD tools expose this directly (GitHub Actions, CircleCI, Vercel, etc.).

In an AI context: check whether development time changes, then inspect review, validation and deployment queues. A stable deployment frequency alone does not locate a bottleneck or establish an AI effect.

Signal to inspect: deployment frequency and Change Failure Rate rise together. Inspect changes and incidents before attributing the cause to review or AI.

2.2 Change lead time

What it measures: Time between a commit and that code in production.

How to measure: Timestamp at commit, timestamp at deployment. The delta is your lead time. Tools like LinearB and Faros.ai automate this from your CI/CD pipeline.

In an AI context: compare coding, waiting, review, validation and deployment time separately. Assistance can change several stages; unchanged total lead time alone does not identify which stage constrains the flow.

Diagnosis: Map your stages explicitly (coding time, waiting for review, waiting for staging, deployment window) to know where the lever is.

2.3 Change failure rate

What it measures: Percentage of deployments that cause a production incident or require a rollback.

How to measure: (Failed deployments) / (Total deployments). “Failed” means requiring a hotfix, rollback, or incident response. Define that clearly before measuring.

In an AI context: plausible code can contain behavioral defects. Measure failed deployments and review evidence over comparable windows; no general 6-to-12-month rise in CFR is established here.

Recommendation: where attribution is reliable, compare CFR by recorded AI assistance within comparable change classes. Include unknown attribution and sample sizes. Task risk, review practices and deployment policy can explain a difference; it does not identify the cause.

2.4 Failed deployment recovery time

What it measures: Time to recover from a failed deployment that requires immediate intervention.

How to measure: Link the failed deployment, incident, intervention, and recovery timestamp. Exclude failures unrelated to a software change.

In an AI context: AI-assisted diagnosis can reduce recovery time when the model can inspect deployment metadata, logs, traces, and code. Measure the recovery-time distribution, false diagnoses, and human intervention time. A demo is not evidence of improvement.

2.5 Deployment rework rate

What it measures: The ratio of deployments that are unplanned work caused by a production incident.

How to measure: Divide unplanned corrective deployments by total deployments over the same period, with each corrective deployment linked to its incident.

In an AI context: Generation speed can hide hotfixes, rollbacks, and follow-up deployments. Attach token cost and human review time to this rework. A higher deployment frequency with a rising rework rate is not a throughput improvement.

2.6 From four keys to five metrics

The official history records two changes that many dashboards still miss. DORA replaced MTTR with Failed Deployment Recovery Time in 2023, then added Deployment Rework Rate in 2024. The current model therefore has five metrics.

The 2025 report also replaced the old Elite, High, Medium, and Low clusters with seven broader team archetypes. Historical tier tables are not current universal targets. Compare one service against its own baseline and improve its main constraint.

3 Beyond DORA: The SPACE framework

DORA measures the delivery system; SPACE measures the people inside it. Both are necessary, and neither is sufficient alone.

SPACE was developed by researchers from GitHub, Microsoft, and the University of Victoria (published 2021). It covers five dimensions.

3.1 Satisfaction and Well-being

Are developers satisfied with their work, their tools, and their processes? Are there burnout signals?

Measure with: a quarterly developer experience survey (5 to 8 questions, anonymous). Track the trend over time, not the absolute score. A team at 3.2/5 moving to 3.8/5 over 6 months is in a better position than a team stuck at 4.0/5.

3.2 Performance

Does the work delivered function as intended? Does it meet quality and reliability expectations?

Measure with: Change Failure Rate (overlap with DORA), Bug Escape Rate (bugs reaching production divided by total bugs), and customer satisfaction on specific features.

3.3 Activity

What volume of work is produced?

Measure with: deployment frequency, throughput (features delivered per cycle), PR merge rate.

Critical note: Activity is the easiest dimension to measure and the easiest to optimize perversely. High commit counts, high PR volumes, high deployment frequency can coexist with very low actual value delivered. Activity metrics are inputs, not outcomes.

3.4 Communication and collaboration

Is knowledge flowing? Are teams unblocked and connected?

Measure with: PR review latency (time between opening and first review), cross-team dependency resolution time, new contributor onboarding time.

3.5 Efficiency and flow

Can developers do deep work without constant interruption? What friction exists in the development process?

Measure with: self-reported flow state frequency (in your developer survey), context-switching frequency, unplanned versus planned work ratio.

3.6 The velocity trap

Teams can reach “High” on DORA deployment frequency while scoring poorly on satisfaction, well-being, and efficiency. More deployments, but developers working evenings to meet sprint commitments, skipping design discussions because AI makes coding fast enough to skip planning, accumulating cognitive debt from AI code reviews they don’t fully understand.

SPACE captures these signals where DORA doesn’t. Using both frameworks together gives you the complete picture, and a divergence (DORA strong, SPACE weak) is an early warning signal of future DORA degradation: exhausted teams eventually deliver more slowly.

> 📌 **Recommended cadence**
> **DORA**: monthly leadership review. System health, delivery pipeline performance.
>
> **SPACE** (satisfaction and efficiency dimensions): quarterly review. Human health, sustainable pace, skill development.

4 AI-specific metrics

Standard frameworks weren't designed with AI-assisted development in mind. These metrics fill the gap.

4.1 % AI-assisted code

The proportion of committed code that was AI-generated or AI-assisted. Available in Anthropic Contribution Metrics (Team and Enterprise plans), the GitHub Copilot dashboard, and similar tools for other AI coding assistants.

Why it matters: Provides context for everything else. A 5% CFR increase is a different signal if AI assists 10% of your code versus 80%. Track this as the denominator for all quality metrics. This number typically rises over time as adoption grows; benchmark it quarterly.

4.2 CFR: AI code vs. manual code

Separate your Change Failure Rate by code origin: AI-generated commits versus manually written commits. Most enterprise AI coding tools can tag commits or PRs.

Compare comparable services and changes, with adequate sample sizes and uncertainty. A CFR difference does not identify its cause: task risk, attribution errors, review practices and deployment policy may all contribute.

4.3 Review time: AI PRs vs. manual PRs

Compare the average review time (opening to merge) for AI-generated PRs versus manually written PRs. If AI PRs are merged significantly faster than manual ones, you may have a rubber-stamping problem.

AI-generated code requires at least as much scrutiny at review as manually written code, possibly more: it can be wrong in non-obvious ways with high apparent confidence. Investigate unexpectedly short reviews in relation to change size, risk and review evidence; there is no universal percentage threshold.

4.4 Developer code understanding

A binary qualitative signal: during a PR review, can the author explain their AI-generated code in their own words, not just what it does but why it does it that way?

Track this informally through your code review culture. If reviewers start noticing that authors can't explain their AI-generated submissions, that's a skill atrophy signal that can later appear as higher CFR and longer failed deployment recovery time.

4.5 PR comprehension time

A proxy for code clarity and maintainability: ask reviewers to report how long it took them to understand what a PR does (before they can assess whether it's correct). Track the median across your team.

Rising comprehension time suggests code is becoming more complex or less well-organized over time, regardless of who wrote it. AI-generated code can inflate this metric by producing syntactically dense implementations that are harder to reason about than simpler, more explicit alternatives.

5 Agentic metrics: What DORA doesn't measure

DORA and SPACE were designed for deterministic software systems. Agents introduce non-determinism, probabilistic quality, and failure modes that fall through every existing metric category. Three groups of metrics fill that gap.

5.1 Why standard DORA is insufficient for agent workflows

DORA measures pipeline health, not output correctness. A study of 39 agent frameworks and 439 agentic applications (arXiv 2509.19185) found that 70% of test effort concentrates on deterministic components (tools, workflows) while less than 5% covers the LLM Plan Body, the central reasoning component most likely to produce incorrect results. Most DORA tooling has the same blind spot.

DORA doesn't capture the cost of the verification loop either. Agent-generated PRs require at least as much review scrutiny as manually written code, and in practice more, because subtle behavioral errors can appear in syntactically correct code. Standard deployment frequency and lead time metrics look identical whether review is thorough or rubber-stamped.

5.2 Group 1: RCT-Verifiable metrics

These follow distinct methods: METR is randomized; DeputyDev is a cohort study. Run them on real tasks, not synthetic benchmarks. They give you numbers calibrated to your own team rather than vendor estimates.

Metric	What it measures	Published baseline
Task completion time, AI vs. no AI	Actual speedup or slowdown for your task type	METR Study 1 (July 2025, arXiv 2507.09089): 19% longer completion time for experienced developers on complex open-source repos. Perception gap: +39 points, developers believed they were 20% faster
PR cycle time, AI-assisted vs. baseline	Pipeline throughput change	DeputyDev (arXiv 2509.19708): –31.8% PR cycle time (p=0.0018), n=300 engineers, 12 months

Metric	What it measures	Published baseline
Pass rate on PRs with executable tests as oracle	How often agent-generated code actually passes functional validation	c-CRAB (arXiv 2603.23448, March 2026): Claude Code 32.1% pass rate on real PRs. Union of four tools: 41.5%. These results are specific to the benchmark and tested systems, not a ceiling on code review quality

The METR result concerns Cursor Pro and Claude 3.5/3.7 Sonnet on complex existing repositories in early 2025. Keep those historical model names and task conditions when interpreting it. Different study designs and self-reports cannot be combined into a universal productivity estimate.

5.3 Group 2: Agentic pipeline metrics

These require instrumentation via Langfuse, Arize Phoenix, or AWS Bedrock AgentCore. They do not replace DORA; they sit alongside it as a layer specific to non-deterministic systems.

Metric	What it measures	Reference
Spec quality score	Completeness and precision of specs before agent execution, a leading indicator for output quality	No standard rubric yet; define internally. Factory.ai's pre-implementation validation contracts are the closest documented proxy
Validation contract pass rate	Percentage of agent-generated implementations that pass pre-defined behavioral contracts, measured before human review	Factory.ai Missions pattern: 81 problems detected in a Slack clone from spec alone, generating 34% of implementation work as fix features
Agent task completion rate	Tasks the agent completes without human correction, expressed as a percentage	Instrument via harness logs. Anthropic's internal data shows the 99.9th-percentile task duration grew from 25 to 45 minutes between October 2025 and January 2026, indicating agents are handling more complex tasks
Code review recall	Share of reference defects correctly detected, using an explicitly defined benchmark denominator	Code Review Bench (Martian, March 2026, 200,000+ open-source PRs): Augment Code 62.8% recall, GitHub Copilot 53.3% recall, Graphite 75% precision but only 8.8% recall
Cost per accepted task	Full attempt, review, rework and recovery cost divided by tasks accepted under the stated criteria	No industry benchmark published yet. Track manually: tokens consumed, cost per model call, human review hours per task completion

Metric	What it measures	Reference
Tokens per feature	Average tokens consumed per merged feature, crossed with Jira or Linear ticket boundaries. A better signal than tokens per request because it accounts for session count variation per feature	No industry benchmark. Track via ccboard's project leaderboard (tokens per session times sessions per feature). Establish a local baseline before optimizing; no universal token range is established here

5.4 Test a harness before standardizing it

A harness trial should compare two or three candidates on the same real tasks, with the same instructions and model where possible. Write pass criteria before the trial, isolate each candidate in its own worktree, and retain the traces that explain the result. A public benchmark can inform the shortlist; it cannot replace that local test.

Measurement	Decision question
Human verdict	Is the change acceptable without unplanned rework?
Interventions	How many human corrections or redirects were needed?
Plan drift	Did the work stay within the stated scope and contract?
Wall time	What elapsed time passed from launch to verified result?
Accepted-task cost	What cost belongs to tasks the team actually accepted?
Recovery	What remains after a failure, interruption, or rejection?
Setup friction	Which permissions, dependencies, and operations remain to maintain?

The Agent Harness Map test protocol connects these measures to the choice between a runtime, repository contract, and orchestrator. They complement DORA; they do not measure people.

5.5 Group 3: Agent governance metrics

Sourced from Strata Identity Research 2026 and CSA/Zenity 2026. These reflect the state of organizational maturity rather than technical performance.

Metric	Target state	2026 baseline for context
Agents with named human sponsor	Every active agent has an identifiable owner	Only 28% of organizations can link agent actions to a human sponsor in all environments (Strata 2026)

Metric	Target state	2026 baseline for context
Real-time agent inventory coverage	All active agents appear in a central registry	21% of organizations maintain a real-time inventory (Strata 2026)
Credential rotation frequency	Agent credentials rotate at least every 90 days	44% of organizations still use static API keys for agent authentication (Strata 2026)
Permission violations detected	Violations detected as a share of estimated total violations	53% of organizations have experienced an agent incident in the past 12 months; 58% took more than 5 hours to detect it (CSA/Zenity 2026)

5.6 The heavy-user review time contradiction

Measure time spent reviewing generated and manually written changes, together with change size, risk, defects and rework. A higher review load may reflect more output, harder tasks or process changes; without a controlled comparison it does not establish the cause.

Break large reviews into coherent changes and monitor comprehension and missed defects. Historical review studies can motivate this practice, but a fixed line count or session duration is not a universal quality boundary. See The Attention Cost of the Review Shift for context.

5.7 The Uplevel Copilot study

Uplevel's study of Copilot adoption (uplevelteam.com, "genai-developers") found no significant change to coding speed, PR cycle time, or throughput after teams adopted Copilot, alongside a 41% increase in bug rate within PRs. Their "Sustained Always On" proxy for burnout risk, built from after-hours and weekend activity patterns, also declined more for developers without Copilot than for those with it over the same period.

Treat vendor observations as context rather than proof that AI caused changes in bug rate, throughput or burnout. Compare the result with your own task mix, review effort and production outcomes.

5.8 pass^k for non-deterministic tests

For a stochastic agent workflow, pass^k asks whether all k independent executions of the same task satisfy a fixed evaluator. Rerun the agent from comparable initial conditions, keeping the task and oracle stable. Repeating the same deterministic test suite on one generated

patch measures test repeatability, not agent pass^k. Choose k and the sample size before comparing systems.

6 Product metrics

Engineering metrics measure how code is built. Product metrics measure whether the code actually solves the right problems. Most engineering teams track the former and leave the latter entirely to product managers. That creates a blind spot where a team can ship fast, with strong DORA scores, while the product drifts away from user needs.

6.1 Time-to-value

How long does it take a new user to reach their first success with your product? Define “first success” concretely: first task completed, first item saved, first report generated.

Track this as a median across user cohorts, and watch for regressions after major releases. AI can accelerate your feature delivery rate without improving the new user experience, or even degrading it.

6.2 Feature adoption rate

Among users who could use feature X, what percentage actually uses it within 14 days of release? A feature delivered on time with strong DORA metrics but that nobody uses is still a failed feature.

Segment by user cohort (new versus existing users, different pricing tiers) to distinguish adoption problems from discoverability problems.

6.3 Bug escape rate

Bugs found in production divided by total bugs (pre-production bugs plus production bugs). Formula: `bugs_in_prod / (bugs_before_prod + bugs_in_prod)`.

Define consistent severity, discovery windows and reporting rules before comparing Bug Escape Rate. There is no universal 20% boundary. Investigate trends alongside deployment volume, test coverage and incident impact.

6.4 Feature CSAT

Customer satisfaction score tied to specific features, not the product as a whole. More actionable than NPS: instead of “how likely are you to recommend us,” ask “how useful was feature X for accomplishing task Y” on a 1-to-5 scale.

NPS is useful for brand-level sentiment but too lagging and too broad to drive development priorities. Feature CSAT gives you a signal within 2 to 4 weeks of a release rather than 6 to 12 months.

7 By team size

Different team sizes have different tolerances for measurement overhead. A 5-person team spending 20% of its time on metrics infrastructure is making a bad trade-off. A 25-person team without automated DORA tracking is navigating blind.

7.1 5-person team

Metrics to track:

Metric	How	Frequency
Deployment frequency	Manual count or from CI	Weekly
Cycle time (commit to prod)	Linear or GitHub timestamp diff	Per PR, reviewed monthly
Time-to-value (north star product metric)	Analytics tool (Mixpanel, Amplitude, PostHog)	Monthly
Production bugs per month	Count in your issue tracker	Monthly
Developer satisfaction	Anonymous 5-question form	Quarterly

Tooling: GitHub Insights plus a shared spreadsheet. No need for a dedicated dashboard at this size; the overhead isn't worth it. What matters is the discipline of reviewing these numbers in a monthly retrospective, not the precision of tooling.

> 📌 **The 'we're too small' trap**
> The instinct at this size is often to skip metrics entirely ("we're too small, we know each other, we talk every day"). Resist that. The value of metrics at 5 people is the discipline, not the visibility. Naming a north star metric and checking it every month forces conversations that daily standups don't naturally trigger.

7.2 25-person team

Metrics to track:

Metric	How	Frequency
All 5 DORA metrics	LinearB or Faros.ai automated, after checking definitions	Weekly (automated)
Cycle time by squad (not global)	Same tooling, segmented	Weekly

Metric	How	Frequency
Bug Escape Rate	Issue tracker plus deployment markers	Monthly
Feature CSAT	In-app survey on key features	Per release
% AI-assisted code	Anthropic Contribution Metrics	Monthly
Developer satisfaction	Anonymous 8-question survey	Quarterly
PR review time	GitHub Analytics / LinearB	Weekly
Time-to-value	Analytics tool	Monthly

Tooling: LinearB or Faros.ai for DORA automation, GitHub Analytics for AI contribution data, PostHog or Amplitude for product metrics.

> 📌 **Avoid global averages**
> At 25 people, global averages hide squad-level problems. A team with 3 squads where 80% of incidents come from one squad will show a “Medium” global CFR and miss the signal entirely. Track DORA by squad, not just by organization.
>
> PR review time is particularly revealing at this scale. When the review median exceeds 24 hours, it creates context-switching overhead: developers move to other tasks while waiting, then have to reload context when the review comes back. That reload cost doesn’t show up in any standard metric, but it accumulates across dozens of PRs per week.

8 Vanity metrics to drop

Drop	Replace with	Why
Sprint velocity	Cycle time + deployment frequency	Velocity is manipulable in 2 sprints by changing estimation practices. Cycle time is harder to game.
Lines of code	Bug Escape Rate	LOC measures output volume. With AI, LOC rises automatically. Bug Escape Rate measures output quality.
NPS alone	CSAT + time-to-value	NPS is a lagging brand signal, not an engineering steering metric. Feature CSAT is actionable within weeks.
Commits per day	Lead Time for Changes	Commit frequency measures activity. Lead time measures whether that activity actually delivers value.
Story points	Throughput (features delivered)	Points are defined relative to the team's own baseline and are manipulable by re-pointing. Throughput counts actual deliverables.
Code coverage rate	Mutation testing score + Bug Escape Rate	Coverage tells you tests exist. Mutation testing tells you whether those tests would catch real bugs.

Note on story points in an AI context: if AI generates boilerplate and scaffolding automatically, the effort to implement a "3-point story" has dropped considerably. Teams that haven't recalibrated their pointing will see velocity increases that reflect tool efficiency, not team capacity.

9 The 4-question test

Before adding a metric to your stack, run it through these four questions.

1. Can you act on it within 2 weeks?

If the answer is no, it's a reporting metric, not a steering metric. Reporting metrics belong in quarterly investor decks, not weekly team reviews. "Total API calls since launch" is a reporting metric. "API error rate over the last 7 days" is a steering metric.

2. Does it explain the why, not just the what?

"Churn is at 5%" tells you nothing. "80% of churned users never completed their first workflow" tells you where to look. When evaluating a metric, ask: if this number moves, do I know what to investigate?

3. Is it correlated with a business outcome?

Deployment frequency correlates with revenue in fast-iteration SaaS products. PR review time correlates with developer satisfaction and retention. Feature CSAT correlates with expansion revenue. Lines of code correlate with nothing important.

4. Can it be measured automatically?

If collecting the metric requires manual work, it will be abandoned within 3 months when load increases. Automate or eliminate.

The rule: fewer than 3 "yes" answers, drop the metric. A stack of 5 rigorous metrics is more useful than a stack of 20 poorly defined ones.

9.1 Admit work at verification capacity

The Lean delivery lens follows one request from a user need through review, integration, release and use. Faster generation can move waiting into those later stages. Before starting another task, name the verifier, required evidence and the available review capacity. When that capacity is exhausted, pause admission and finish, narrow or reject existing work.

Track work in progress, queue age, accepted throughput and end-to-end cycle time for the same class of change. Include rejected attempts, false blocks, review and rework. Record the reviewed head, base and acceptance criteria; a changed requirement or combined change requires affected checks to run again. Compare escaped defects and the user outcome over comparable windows before increasing parallelism. This is a control to test locally, not a promised productivity multiplier. The review-admission worksheet provides a concrete experiment.

10 Tooling

Tool	What it does	Best for	Notes
LinearB	DORA automation + cycle time, connected to GitHub + Jira	25+ people	Strong DORA dashboards out-of-box, solid PR analytics
Faros.ai	DORA + custom engineering dashboards, open-source core	25+ people	More configurable than LinearB, heavier setup
GitHub Analytics (Anthropic)	AI contribution metrics (% AI-assisted code, PR attribution)	Any Claude Code team	Enterprise/Team plan required
Sleuth	Deployment tracking + change failure rate, DORA-focused	10+ people	Lightweight, CI/CD-focused, no bloat
Axify	Full engineering metrics suite, DORA + flow + team health	15+ people	Canadian startup, strong SPACE coverage
GitHub Insights	Basic activity metrics, free, native	Any size	Sufficient for 5-10 people, insufficient at scale
Spreadsheet	Manual tracking, always works, zero setup	Fewer than 10 people	The right tool when automated setup overhead isn't justified
LiteLLM + Loki + Tempo + Grafana	Self-hosted usage, cost, and trace aggregation	25+ people, self-hosted requirement	LiteLLM Gateway logs usage server-side with zero client-side setup; Loki ingests session JSONL for file-level detail; Tempo correlates AI sessions with deployment traces via OTel; Grafana panels cover daily spend by team, request volume by model, and budget headroom via PromQL

No tool automatically surfaces the AI-specific metrics described above (CFR by code origin, review time comparison, comprehension signals). That requires either custom dashboards built on your CI/CD data or manual tracking. GitHub Analytics covers % AI-assisted code; the rest you'll need to wire up yourself. Teams that can't send usage data to a third-party vendor should default to the self-hosted LiteLLM/Loki/Tempo/Grafana path instead of LinearB or Faros.ai.

Avoid tool proliferation. A team with LinearB, Jira, GitHub Insights, and two separate analytics tools will spend more time reconciling numbers than acting on them. One DORA tool, one product analytics tool, GitHub Analytics for AI-specific data.

10.1 Broader delivery intelligence platforms

The starter table above covers what most teams reach for first. The 2026 market is wider, and worth a second look if that table doesn't fit your organization's size or constraints.

Tool	What It's For	Notes
DX (getdx.com)	Developer intelligence platform built by researchers associated with the DX Core 4 / SPACE framework lineage	Positioned for larger organizations wanting a research-grounded metrics platform rather than a plug-and-play dashboard
Multitudes (multitudes.com)	Analytics and recommendations built directly on DORA, SPACE, and DevEx signals	Sends "nudges" when a metric indicates a team is blocked or overloaded, rather than just reporting the number
Swarmia (swarmia.com)	Cycle time breakdown (shipped 2022, still active): decomposes PR time into in-progress, waiting-for-review, and waiting-for-merge	Also publishes its own research on the productivity impact of AI coding tools (2025 blog)
Cortex.io	Positions itself as an "Engineering Operations Platform" and publishes its own "Engineering Intelligence Platforms: Top 8 Tools" category guide	Already cited elsewhere in this whitepaper series as a data source for PR size and change failure rate figures; documented here as a delivery-intelligence tool in its own right for the first time
Jellyfish	Positions itself in 2026 as a "software engineering intelligence platform" explicitly built for AI-integrated organizations	Same note as Cortex.io: already cited as a data source elsewhere, now documented as a tool
Oobeya (oobeya.io)	Aggregates 20+ existing DevOps tools into a single layer, rather than being a standalone data source itself	Fits teams already running several disconnected DevOps tools who want one pane of glass instead of another data collector
Hatica (hatica.io)	Markets "gen AI-driven engineering analytics"	Thinner on documented specifics than the other entries in this table: the underlying LLM mechanism behind its analytics claims isn't publicly detailed. Worth a direct vendor conversation before committing

10.2 AI-Generated board narratives

In 2026, some of these platforms now generate a written narrative from the metrics they already compute, rather than leaving that translation to a human, a genuinely new capability. Two concrete examples.

LinearB generates an AI iteration summary from Jira, Git, PR, and deployment data, structured around three axes: what went well, what didn't, and what could improve. It's delivered automatically to Slack or Microsoft Teams at sprint end, replacing the manual retro write-up a tech lead would otherwise produce.

Jellyfish's "AI Executive Report" translates AI-tool adoption data (Copilot, Windsurf, and similar) into three dimensions built for a non-technical executive audience: adoption, output, and impact, the last framed as time saved, money saved, and risk reduced.

> 📌 **What the AI is actually doing**
> In both cases, the generative AI's role is explanation and prioritization of what the underlying analytics already computed, not the computation itself. The pass rate, the cycle time, the CFR split: those numbers come from the same instrumentation covered throughout this whitepaper. The LLM's job is turning a table into three paragraphs a stakeholder will actually read, not producing new statistical insight.

11 Probabilistic delivery forecasting

Every benchmark so far in this whitepaper answers “how are we doing.” This section answers a different question: “when will this ship,” and it answers it with a probability distribution instead of a single date.

11.1 Why a point estimate misleads

Traditional forecasting takes your velocity (story points or throughput per sprint) and divides it into remaining scope to produce a single date. That date carries false precision, implying a certainty the underlying data never had. Two teams with identical average velocity can have wildly different variance, and variance is exactly what a point estimate throws away.

Monte Carlo forecasting samples historical throughput or cycle-time observations to produce a distribution conditional on the model and data. A stated probability needs backtesting and sufficiently comparable conditions; simulation alone does not guarantee calibrated confidence.

11.2 Tools

ActionableAgile (actionableagile.com) is a flow and predictability-focused analytics product, licensed via 55 Degrees AB, built around Monte Carlo forecasting. Its “Portfolio Forecaster” extends the same technique across multiple initiatives at once, useful when a board is asking about predictability across a roadmap rather than a single deliverable.

Nave (getnave.com) runs Monte Carlo simulation alongside a “Cycle Time Histogram” view.

> 📌 **The one requirement that makes or breaks this**
> Choose a data window representative of the current workflow and evaluate forecast calibration on held-out outcomes. A fixed count of 20 or 30 completed items is not a general sufficiency guarantee. Changes in scope, team or process can invalidate the historical distribution.

11.3 What this does not fix

Monte Carlo forecasting is not a fix for an unstable or unpredictable delivery system. If your cycle times swing wildly because scope keeps changing mid-sprint, or because the team

composition just shifted, the simulation will faithfully replicate that unpredictability as a wider distribution. It won't narrow the range for you. A wide, honest 80% confidence interval is still more useful than a false point estimate, but it's a symptom report, not a cure. Fix the underlying instability first; the forecast will narrow on its own once the process does.

12 Implementation roadmap

The most common failure mode in metrics programs is trying to instrument everything at once.

12.1 Phase 1: Instrument DORA

Connect your CI/CD pipeline to a metrics tool. For most teams, that means connecting GitHub Actions or its equivalent to LinearB, Faros, or Sleuth. Automate Deployment Frequency and Change Lead Time first. Change Fail Rate, Failed Deployment Recovery Time, and Deployment Rework Rate require deployment-to-incident linkage.

Output: a live dashboard showing at minimum Deployment Frequency and Lead Time for Changes. Your first baseline numbers.

12.2 Phase 2: Baseline and targets

Once you have representative observations, document the baseline, sample size and uncertainty. The temptation here is to compare immediately to industry benchmarks. Resist that. Set internal improvement targets first: “reduce Lead Time by 20% over the next quarter” is more actionable than “reach the High tier.” Your context (tech stack, deployment environment, team size, product type) affects what’s achievable more than any industry benchmark.

Launch your first developer satisfaction pulse (5 questions, anonymous, 10 minutes to build in Google Forms or Typeform). That’s your SPACE baseline.

12.3 Phase 3: Product and AI metrics

Once DORA is stable and automated, add product metrics (time-to-value, Feature CSAT) and AI-specific signals (% AI-assisted code, CFR by origin). These require more setup (product analytics instrumentation, PR tagging conventions) but are worth the investment once your DORA foundation is solid.

Review the full stack quarterly and prune ruthlessly. Any metric that hasn’t guided a decision in the last 3 months is a reporting metric disguised as a steering metric. Cut it.

13 Reporting delivery capacity to a skeptical board

Everything above this section assumes the audience is your own team or your engineering leadership chain. A board that has watched past estimates slip needs a different approach, because the problem it's raising usually isn't the one a dashboard answers.

13.1 The diagnostic: Doubt is a trust problem, not a data problem

When a board starts doubting a team's delivery capacity after past estimates have slipped, adding more delivery dashboards rarely fixes it on its own. The board's doubt is almost always about trust and visibility, about what caused the slip and whether it will happen again. It is much less often about the data itself: board members generally don't disbelieve the velocity number on the screen, they disbelieve the story that number is supposed to support.

> 📌 **An evidence gap, stated plainly**
> This whitepaper does not establish whether delivery-intelligence tooling repairs executive trust. That's a gap in the field, not a gap in this whitepaper. Nothing here, and none of the tools covered earlier, comes with proof that it fixes a board's confidence. What follows reframes the conversation into one a board can act on, not a guaranteed remedy.

13.2 Bring scenarios, not a velocity chart

A single velocity number invites a single question, whether the team can go faster. That's rarely the question worth answering, and it's an easy one to lose regardless of the real numbers behind it.

Bring 2 to 3 named delivery scenarios instead, each with a concrete timeline, a rework or cost estimate if sequencing gets compressed, and the business impact of picking that option. "Ship the full scope in Q3 at current pace" versus "compress two workstreams into Q2 at an estimated higher defect rate and additional engineer-weeks of rework" versus "cut scope and ship on the original date" is a decision a board can actually make. "Can the team go faster" is not.

13.3 Cap strategic objectives, don't roadmap every feature

A fully detailed, multi-quarter feature roadmap looks like rigor and produces the opposite. Every feature-level commitment beyond roughly one quarter out is a precision claim your delivery system almost certainly can't back, and it erodes trust the moment reality diverges from the plan, which it will.

Cap the number of board-level strategic objectives instead: a small, named set, reviewed quarterly. Objectives survive a schedule slip better than feature line items do, because an objective describes an outcome the team is still steering toward even when the specific path changes.

13.4 Separate committed scope from best-effort scope, and track the hit rate

Draw an explicit line between what the team commits to and what it will attempt on a best-effort basis. Then track, over time, how often committed scope actually ships as committed.

That hit-rate track record becomes the trust-rebuilding metric in its own right. A board that sees "when this team commits, it delivers" repeated release after release cares about that pattern more than it cares about any single throughput or velocity number. Trust rebuilds through a track record, not through a chart.

13.5 Align stakeholders before the room, not in it

A plenary board meeting is a poor venue for first-time alignment on a contentious delivery question. Before walking in with scenarios and numbers, talk to board members individually and understand what's actually driving each one's doubt: comparison to other teams in the portfolio, external business pressure unrelated to engineering, or a specific past miss they haven't let go of. Those three causes call for different framing, and a plenary meeting doesn't give you the space to diagnose which one you're dealing with in the room.

13.6 Where forecasting and AI narratives fit, and where they don’t

The probabilistic forecasting tools covered earlier in this whitepaper give you the honest range behind each scenario above: “80% confidence between these two dates” is a more defensible number to bring into the room than a single point estimate, provided your delivery system is stable enough for the forecast to mean something. The AI-generated board narratives from tools like LinearB and Jellyfish reduce the manual effort of translating sprint data into a written summary a non-technical board member can read.

Neither substitutes for the harder work above. A better forecast and a better-written summary still won’t cap your roadmap, separate committed from best-effort scope, or have the individual conversations that surface what a specific board member actually doubts. Those three remain a management job, not a tooling job.

14 Summary: The complete stack

📌 **Minimal stack by maturity phase**

Phase 1: Foundation

- Deployment frequency
- Lead Time for Changes
- Developer satisfaction (baseline)

Phase 2: Quality signal

- Change Failure Rate (global + by AI/manual origin)
- Failed Deployment Recovery Time
- Deployment Rework Rate
- Bug Escape Rate
- % AI-assisted code

Phase 3: Full steering

- Time-to-value
- Feature CSAT
- PR review time (AI vs. manual)
- PR comprehension time
- Developer satisfaction (recurring)

The temptation is to want everything from the start. A metrics stack, like a codebase, accumulates debt when it grows faster than the team's capacity to act on it. Build it in phases, prune regularly, and keep the 4-question test as a permanent filter.

14.1 PR audit trail

For regulated environments and compliance-conscious orgs, a PR audit trail captures four things at PR creation: the tool call log (which tools Claude used and on which files), the list of files modified with before/after line counts, session metadata (session ID, timestamp, Claude Code version, model), and a hash of the active CLAUDE.md rules. A PreToolUse hook writes this to a daily JSONL log, a GitHub Actions step uploads it as a 90-day artifact on each PR, and a periodic compliance script aggregates tool usage and unique files touched across a date range. This gives auditors a structured "what did Claude do during this change" answer without relying on session memory.

15 See also

- Team Metrics: Reference Guide: full version with dashboards and additional resources
- Session Observability: Claude Code session monitoring, cost tracking, usage patterns
- AI Traceability: AI contribution auditing, attribution, and compliance
- Choosing Your Adoption Approach: Claude Code team adoption patterns, CLAUDE.md configuration strategies

16 The series

This whitepaper is part of the **Claude Code Ultimate Guide** series, a comprehensive resource for mastering Claude Code from installation to production deployment.

13 whitepapers in the series:

- **WP00** Introduction & Fundamentals
- **WP01** Effective Prompts
- **WP02** Customization & Configuration
- **WP03** Production Security
- **WP04** Internal Architecture
- **WP05** Team Adoption
- **WP06** Privacy & Compliance
- **WP07** Condensed Reference Guide
- **WP08** Agent Teams & Sub-Agents
- **WP09** Learning with AI (UVAL)
- **WP10** AI Budget & ROI
- **WP11** Leading a Team in the Age of AI
- **WP12** Agent Engineering

Full series available at cc.bruniaux.com/whitepapers/